Name : Sahil Jakhariya
Div : D15C
Batch : C
Roll No : 63

MLDL Experiment 03

Aim: Apply Decision Tree and Random Forest for classification tasks.

Decision Tree:

1. Dataset Source:

- **Dataset Name: Mushroom Classification**
- **Dataset Name:** Mushroom Classification Dataset
- **Source:** Kaggle
- **Description:**
  This dataset contains information about various mushroom species and their physical characteristics. The goal is to predict whether a mushroom is **edible or poisonous** based on its features.

2. Dataset Description:

The Mushroom dataset is used for binary classification. It consists of multiple categorical features describing the physical attributes of mushrooms.

- **Type:** Classification Dataset
- **Number of Records:** ~8000 samples
- **Target Variable:**
  - class
  - Categories: **Edible (e)** and **Poisonous (p)**

Features include:

- Cap shape
- Cap color
- Odor
- Gill size
- Gill color
- Stalk shape
- Spore print color
- Habitat
- And other categorical attributes

All features are categorical and were encoded into numerical values for model training.

3. Mathematical Formulation of the Algorithm:

A. Decision Tree

A Decision Tree is a supervised learning algorithm used for classification and regression tasks. It splits the dataset into subsets based on feature values.
The quality of a split is measured using:

Gini Index:

$$Gini = 1 - \sum(p_i)^2$$

Entropy:

$$Entropy = -\sum p_i \log_2(p_i)$$

Where p_i is the probability of a class.
The algorithm selects the feature that provides the highest information gain.

B. Random Forest

Random Forest is an ensemble learning method that builds multiple decision trees using random subsets of data and features.

- Each tree gives a prediction
- Final output is decided by **majority voting**

This improves accuracy and reduces overfitting.

4. Algorithm Limitations:

Decision Tree

- Prone to overfitting
- Sensitive to small changes in data

Random Forest

- Computationally expensive
- Less interpretable compared to a single decision tree

5. Methodology / Workflow:

1. Load the dataset
2. Encode categorical features into numerical values
3. Split the dataset into training and testing sets
4. Train Decision Tree classifier
5. Perform hyperparameter tuning
6. Train Random Forest classifier
7. Perform hyperparameter tuning
8. Evaluate models using:
    - Accuracy
    - Confusion Matrix
    - Classification Report

6. Performance Analysis:

- **Decision Tree Accuracy:** ~100%
- **Random Forest Accuracy:** ~100%

The models achieved very high accuracy due to the clean and well-structured nature of the dataset.

Random Forest performs slightly better in general due to:

- Reduced overfitting

- Better generalization

7. Hyperparameter Tuning:

Decision Tree Parameters:

- max_depth
- min_samples_split

Random Forest Parameters:

- n_estimators
- max_depth

Hyperparameter tuning improves model performance and stability.

Code And Output:

```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from sklearn.preprocessing import LabelEncoder

uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])

le =LabelEncoder()
```

```
for col in df.columns:
    df[col] = le.fit_transform(df[col])
X = df.drop("class", axis=1)
y = df["class"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
print("===== Decision Tree =====")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
cm = confusion_matrix(y_test, y_pred_dt)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Decision Tree Confusion Matrix")
plt.show()
plt.figure(figsize=(15,8))
plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=le.classes_,
```

```
  filled=True,
    max_depth=3
)
plt.title("Decision Tree Visualization")
plt.show()
params_dt = {
    "max_depth": [3, 5, 10],
    "min_samples_split": [2, 5, 10]
}
grid_dt = GridSearchCV(DecisionTreeClassifier(random_state=42), params_dt, cv=5)
grid_dt.fit(X_train, y_train)
best_dt = grid_dt.best_estimator_
y_pred_best_dt = best_dt.predict(X_test)
print("===== Tuned Decision Tree =====")
print("Best Parameters:", grid_dt.best_params_)
print("Accuracy:", accuracy_score(y_test, y_pred_best_dt))
cm = confusion_matrix(y_test, y_pred_best_dt)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Tuned Decision Tree Confusion Matrix")
plt.show()
plt.figure(figsize=(15,8))
plot_tree(
```

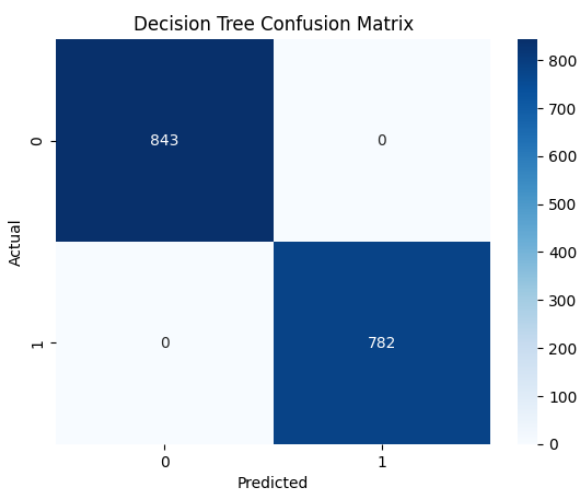
```
    best_dt,
    feature_names=X.columns,
    class_names=le.classes_,
    filled=True,
    max_depth=3
)
plt.title("Tuned Decision Tree Visualization")
plt.show()
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
print("===== Random Forest =====")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
cm = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(cm, annot=True, fmt="d", cmap="Greens")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Random Forest Confusion Matrix")
plt.show()
params_rf = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, 10]
}
```

```
grid_rf = GridSearchCV(RandomForestClassifier(random_state=42), params_rf, cv=5)

grid_rf.fit(X_train, y_train)

best_rf = grid_rf.best_estimator_

y_pred_best_rf = best_rf.predict(X_test)

print("===== Tuned Random Forest =====")

print("Best Parameters:", grid_rf.best_params_)

print("Accuracy:", accuracy_score(y_test, y_pred_best_rf))

cm = confusion_matrix(y_test, y_pred_best_rf)

sns.heatmap(cm, annot=True, fmt="d", cmap="Greens")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Tuned Random Forest Confusion Matrix")

plt.show()
```

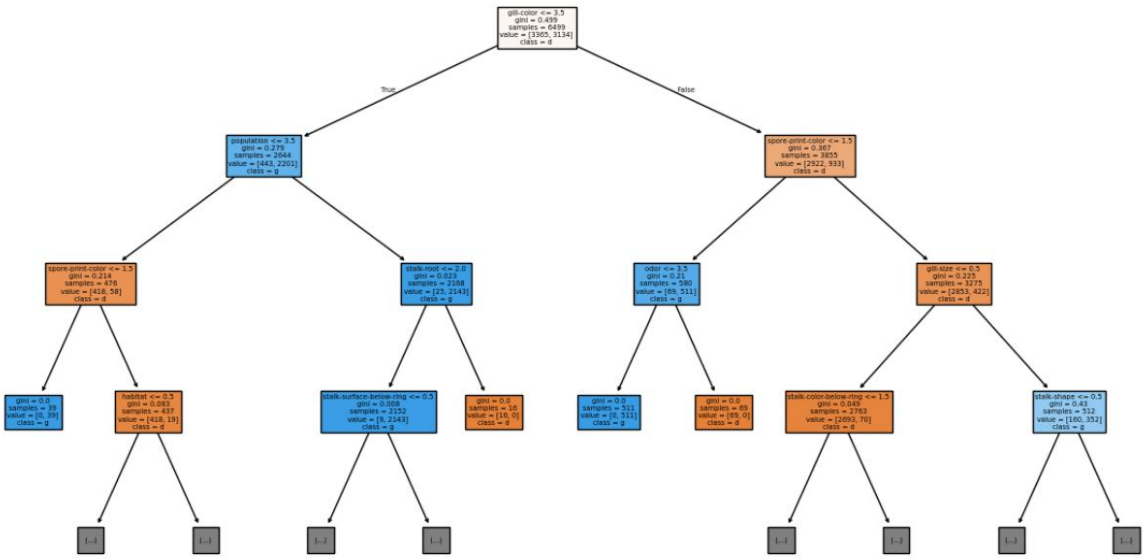
Choose Files mushrooms.csv

mushrooms.csv(text/csv) - 374003 bytes, last modified: 3/21/2026 - 100% done

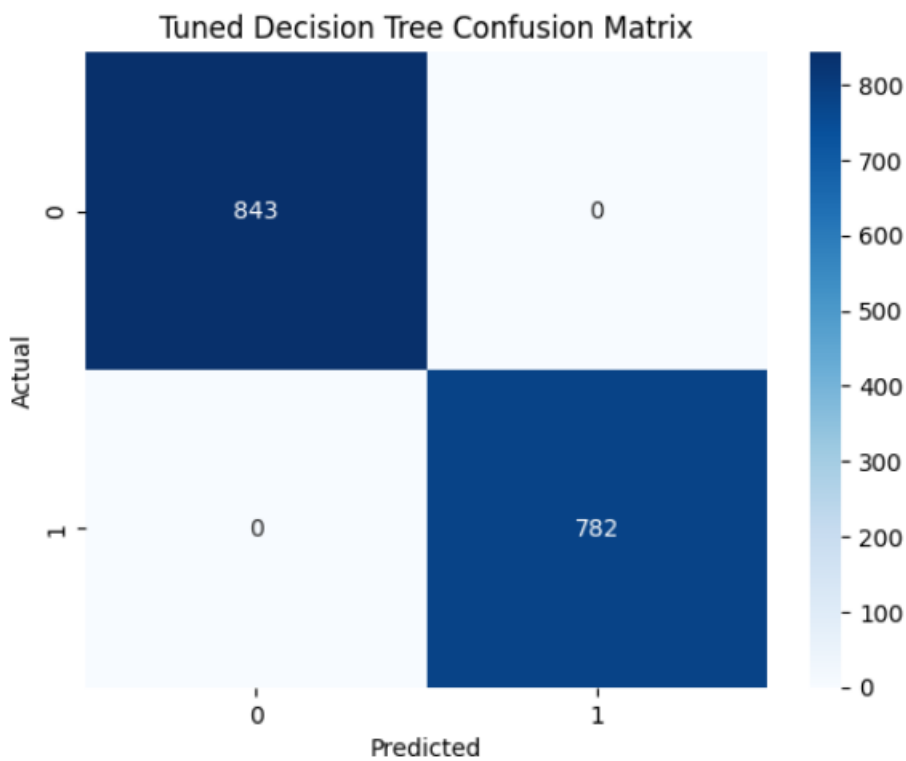
```
Saving mushrooms.csv to mushrooms (1).csv
===== Decision Tree =====
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       843
           1       1.00      1.00      1.00       782

    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625
```



Decision Tree Visualization

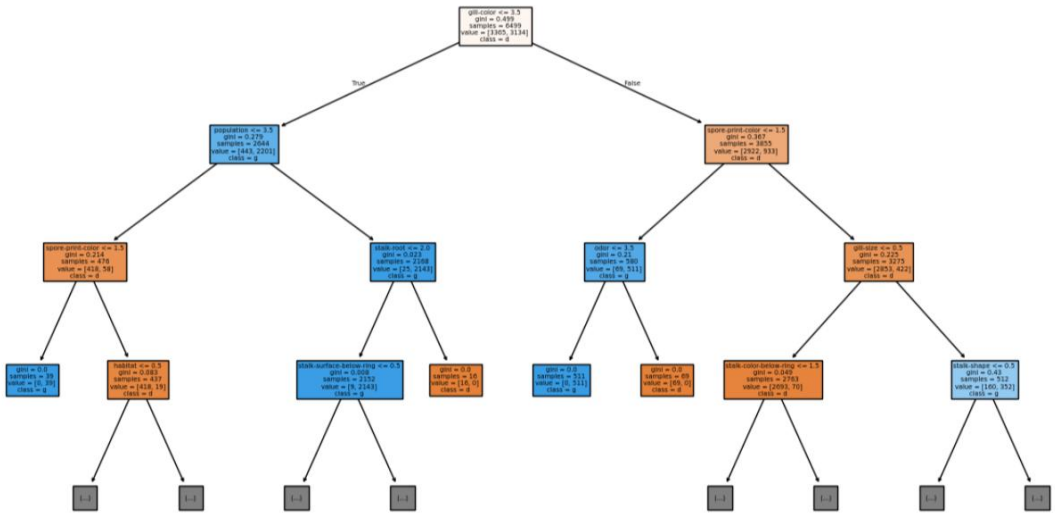



```
===== Tuned Decision Tree =====
Best Parameters: {'max_depth': 10, 'min_samples_split': 2}
Accuracy: 1.0
```

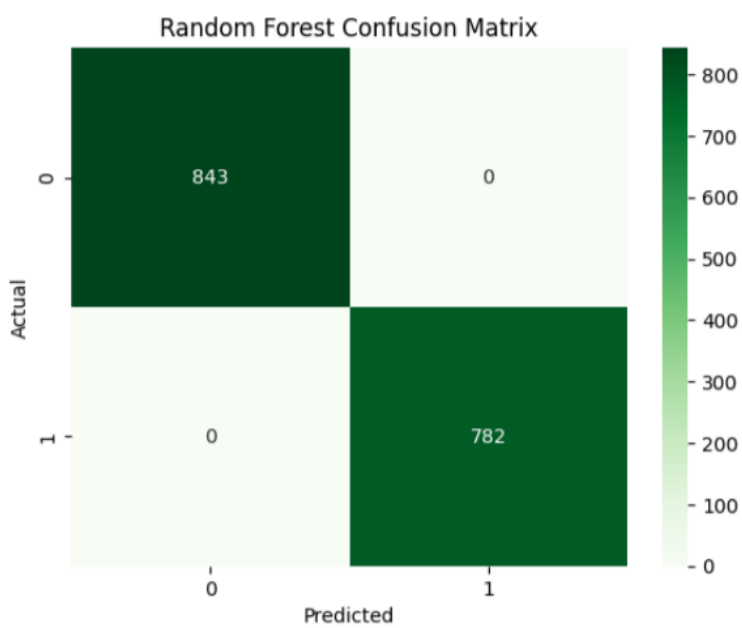


...

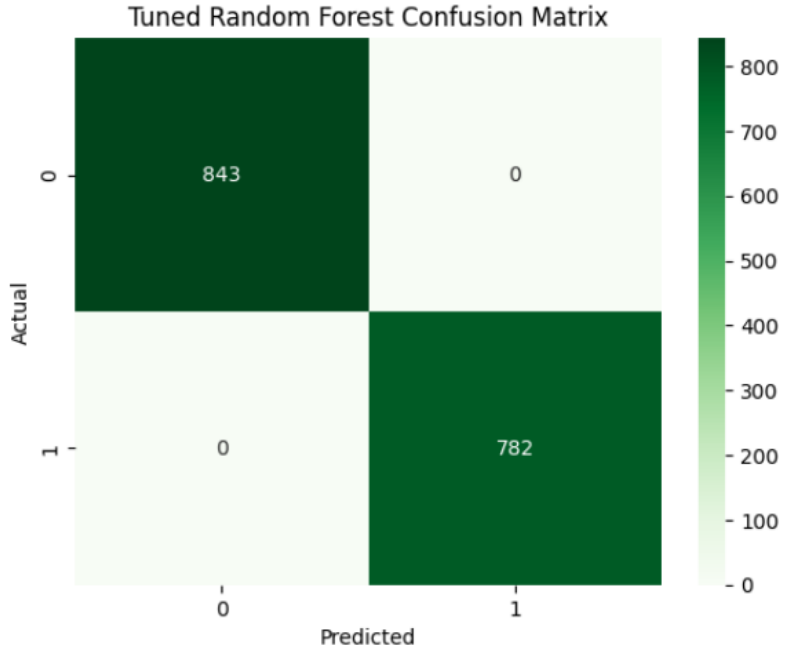
Tuned Decision Tree Visualization




```
===== Random Forest =====
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       843
           1       1.00      1.00      1.00       782

    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625
```



```
===== Tuned Random Forest =====
Best Parameters: {'max_depth': 10, 'n_estimators': 50}
Accuracy: 1.0
```



Interpretation:

Decision Tree

- The tree structure shows how features like **odor, cap shape, and gill size** influence classification.
- Each node represents a decision rule.
- Leaf nodes represent final predictions (edible or poisonous).

A shallow tree indicates good generalization, while a deep tree may overfit.

Random Forest

- Combines multiple decision trees
- Reduces overfitting
- Provides more stable and accurate predictions

The confusion matrix shows almost no classification errors, indicating strong model performance.

9. Conclusion:

This experiment demonstrated the successful application of Decision Tree and Random Forest algorithms on the Mushroom dataset.

Both models effectively classified mushrooms as edible or poisonous based on their features.

Random Forest showed slightly better performance due to ensemble learning, making it more robust.

This approach can be useful in real-world applications where accurate classification is critical.